

ARM-450 — Mistakes, Corrections and Lessons

- Date: 2026-08-14
- Purpose: an honest record of every error found in this project — in the original design, and in my own analysis of it — with the reasoning that exposed each one.
- Why this document exists: you asked for it explicitly. Most of these are not careless slips; they are the standard traps of small-robot design, and each one is worth more as a rule than as a fix.

Part I — Design errors found in the printed arm

M1. The servo mounts were sized for the wrong load — the one that actually broke

What happened: the motor mounts snapped during trials.

The error: they were sized for the 2.08 N·m the arm actually carries. A servo can deliver its stall torque — 2.94 N·m (ST3215), 4.90 N·m (ST3250) — into its own mount during a collision, a commanded step, or when it hits a joint limit.

The numbers: the 4 mm bracket runs at 17–95 MPa depending on where it grips, against a realistically sustainable ~9 MPa for printed PLA under repeated load. It fails under every assumption in that range.

RULE — whenever an actuator can drive its own structure to failure, the actuator's limit IS the design load. Gravity torque tells you what the servo must do. Stall torque tells you what the mount must survive.

The trap inside the trap: upgrading the shoulder to an ST3250 multiplies mount stress 2.36× while changing the load the arm actually carries by nothing at all.

M2. A thrust bearing was used where a moment-carrying bearing was needed — the largest error in the arm

What happened: visible gaps at the joints, wobble, and small oscillations that never settle.

The error: a Ø42 × 3.7 thrust ball bearing carries axial load only — balls run between two flat races, so the joint is free to tilt. But a robot joint's dominant load is a bending moment: 2.08 N·m at the shoulder from everything outboard on a long lever.

The numbers: angular play $\theta = 2c/d$. With effectively no bearing spacing ($d \approx 5$ mm) and 0.2 mm clearance, that is 28.9 mm of wobble at 360 mm reach — larger than every other error source in the arm combined.

RULE — match the bearing to the load direction, not to the shaft diameter. Radial and moment loads need deep-groove pairs, crossed-roller, or four-point-contact bearings. Thrust bearings take thrust.

RULE — moment stiffness scales with bearing spacing SQUARED. Spacing is worth far more than bearing size or count.

M3. The oscillation was mechanical, not a tuning problem

The error: clearance in a position-controlled joint is a dead zone — the servo turns and nothing moves until the slack takes up, then it moves suddenly. A feedback controller pushing against a dead zone produces a limit cycle.

RULE — a limit cycle from backlash cannot be tuned out. Lowering gain makes the arm sluggish and the oscillation smaller, never absent. The fix is mechanical preload.

M4. The 1.1 revision lengthened only one link

The error: link1_base_1.1 grew to 195.1 mm joint-to-joint while link2_base stayed at 97.9 mm.

The numbers: the reachable region of a two-link chain is an annulus whose inner radius is $|L2 - L3|$. That split creates a Ø194 mm dead zone directly in front of the robot, and totals 453 mm — over the 450 mm limit.

What makes this sting: the original links were 97.9/97.9 — perfectly equal, zero dead zone. The revision broke something that was already right.

RULE — for a 2-link arm, $L2 = L3$ eliminates the workspace dead zone, and the cost is almost nothing. The optimisation study (§ below) shows shoulder torque is nearly flat in the split, because the dominant term is the payload at a fixed reach. Equal links are free.

M5. 450 / 6 — dividing the length budget evenly across all six segments

The error: an early sketch worked toward six equal 75 mm segments. That spends length on things that buy no reach (base height, wrist stack, tool) and starves the two links that do — roughly 225 mm of reach from a 450 mm arm.

RULE — the division that should be equal is $L2$ vs $L3$, not all six segments.

M6. Sharp internal corners in loaded brackets

Every reentrant corner is a crack starter. A printed "sharp" corner still has the nozzle radius (~ 0.2 mm), which at $t = 4$ mm is $r/t = 0.05 \rightarrow Kt \approx 2.6$. A 2 mm fillet gives $r/t = 0.50 \rightarrow Kt \approx 1.3$.

RULE — every internal corner in a load path gets $r \geq 0.5 t$, minimum 2.0 mm. It costs nothing in mass or print time and cuts peak stress $\sim 2.3\times$.

M7. An open (notched) section carrying torque

The U-channel motor fixer has a notch cut through it. Open sections are catastrophically poor in torsion: $J \approx 1,730 \text{ mm}^4$ open vs $\approx 189,500 \text{ mm}^4$ closed — $110\times$.

RULE — closing a section is the cheapest stiffness available anywhere in mechanical design.

A bridging strap or bolted cap is usually enough.

M8. Mismatched shell pair

link1_cover (envelope 196.5 mm) did not pair with link1_base (147.5 mm), while link2_cover (146.5) paired correctly with link2_base (147.5) — a leftover from an intermediate revision. You have since fixed this in the new assembly.

RULE — when a clamshell exists in several generations, check that base and cover come from the same one. It is also a strong candidate cause of URDF export failures.

M9. An unconstrained body left floating in the assembly

Found: 2026-08-14, in the rebuilt robotic_arm_assembly.stl.

The base-to-arm mismatch was genuinely fixed — those two groups now mate with zero gap. But one yoke-class component sits 429 mm away in Y, unconstrained. It alone inflates the assembly envelope from ~440 mm to 659 mm.

RULE — check an assembly's bounding box against the design intent before exporting. An envelope 50 % larger than the arm's own length is a free, instant signal that something is unmated. It costs one measurement and catches exactly this class of error.

This is also a strong candidate for the URDF export failures: an exporter walking the assembly tree hits a body with no mate and either drops it or emits a disconnected link.

Part II — Errors in my analysis

These were found by adversarial review and then verified independently. Rev 1 of the analysis report had already been issued when they were caught.

A1. I bent the link about the wrong axis — the significant one

What I did: computed I about the 29.0 mm dimension $\rightarrow I = 39,899 \text{ mm}^4$.

Why it was wrong: the ST3215 is 24.7 mm thick and the link's split-normal cavity is $29.0 - 2(2.0) = 25.0 \text{ mm}$ — a deliberate 0.3 mm clearance fit. The servo output axis is normal to its 45.2×37.8 face, so the joint axis lies along the 29.0 mm direction. That axis is horizontal, therefore 47.5 mm is the vertical bending depth.

Correct value: $I = 87,514 \text{ mm}^4$. Every droop figure in rev 1 was $2.19\times$ too pessimistic.

LESSON — the servo's own fit inside the shell told me the bending orientation, and I did not look. When a part's dimensions include a 0.3 mm clearance on a 25 mm cavity, that is not a coincidence, it is the designer telling you how the part is used.

A2. I claimed an open-section bending penalty that does not exist

What I did: stated an unbolted seam costs $4\times$ in bending, by applying a stacked-section rule.

Why it was wrong: the split plane is parallel to the bending plane, so the two half-shells sit side by side about the same neutral axis and their second moments simply add.

$I_{\text{open}} = I_{\text{closed}}$. There is no bending penalty at all.

And the correction cut the other way too: the torsion penalty is 215×, not the 126× I reported, because my open_J had double-counted the section width.

LESSON — where a section is split relative to the load matters more than that it is split. The seam sits at the extreme fibres, where bending shear flow is zero. It carries essentially only torsional shear flow.

A3. I omitted J1 from the backlash budget

Three joints swing the tool, not two. Corrected RSS at 0.5°: 4.82 mm, not 3.66 mm.

A4. I used the wrong lever arm for shell twist

A twist about the link's longitudinal axis swings the tool by its perpendicular offset from that axis — not by the wrist length.

A5. I modelled the two links as separate cantilevers

That understates the moment arm roughly 3×. The correct model is one continuous cantilever of length $a = L_2 + L_3 = 290$ mm with a rigid overhang $b = 70$ mm. (Caught and fixed before rev 1 was issued.)

A6. I specified a bearing fit from a generic assumption instead of asking for a measurement

What I did: specified a pocket at $\varnothing 42.15$, based on the usual FDM 0.1–0.4 mm hole undersize.

Why it was wrong: your printer measures 0.02 mm undersize in XY — very well calibrated. My pocket would have left 0.13 mm of clearance → 1.17 mm of wobble, not the 0.36 mm I promised. It would have quietly defeated the entire bearing redesign.

Corrected: model the pocket at $\varnothing 42.00$, so the 0.02 mm shrinkage itself becomes a light 0.02 mm interference press fit.

LESSON — a fit is not a number you can look up; it is a property of a specific machine. The generic value was not merely imprecise, it inverted the outcome. Measure first.

AND — shrinkage is anisotropic. Yours is 0.02 mm in XY and negligible in Z. That is exactly why a bearing bore must print with its axis vertical: the bore is then a circle in the XY plane and shrinks uniformly. Printed on its side it would shrink in XY but not Z and come out oval — reintroducing the very tilt the redesign exists to remove.

A7. I put a cantilever stub arm on the servo collar

My first collar had a stub arm to reach the link — the exact bending topology my own report condemns in M1. Replaced with a bolt flange that sits flat against the shell, putting the fasteners in shear with no bending arm.

LESSON — writing the rule down does not stop you breaking it while modelling.

A8. A degenerate demo case hid a real result

My roll-joint demo applied the tool offset along an axis that happened to point straight down, so the moment computed as exactly zero and appeared to prove roll joints are never loaded. The offset must be perpendicular to both the roll axis and gravity.

LESSON — a result of exactly zero deserves suspicion, not satisfaction.

A9. I built the capsule profile with a non-tangent end boss

Found: 2026-08-17, by the user, on the very first printed pair — visible misalignment of the seam exactly where the straight section meets the end boss. They asked whether it was printer shrinkage. It was not; it was my geometry.

The error: I built the capsule as a rectangle of half-height 23.75 mm unioned with end circles of radius 25 mm. Those are not equal, so the boss circle bulges past the rectangle edge instead of being tangent to it. The two curves meet at a sharp vertex at $x = 7.81$ mm, at a shallow 18.2° included angle.

Why a shallow vertex is dangerous: an offset applied to a face moves that face by the offset. An offset applied at a vertex moves the intersection point by $\text{offset} / \sin(\text{angle})$. At 18.2° :

0.15 mm tongue/groove clearance $\rightarrow 0.15 / \sin(18.2^\circ) = 0.48$ mm along the profile

So a clearance I chose as 0.15 mm became ~ 0.5 mm of positional mismatch at that one corner — visible to the eye, and exactly where the user saw it.

Ruling out shrinkage: their measured 0.02 mm shrink, amplified identically, gives 0.064 mm — 8× smaller than the geometric error and far too small to see.

The fix: make it a true stadium by setting $\text{SEC_H}/2 = \text{BOSS_R} = 25$, so the arc is tangent to the edge and there is no vertex at all. BOSS_R could not be reduced — a Ø42 bearing needs $\geq 21 + 2.4$ mm of wall — so the section rose from 47.5 to 50.0 mm deep. Bonus: I improved from 101,992 to 115,853 mm⁴, 13.6 % stiffer.

LESSON — offsets amplify at shallow-angle vertices, by $1/\sin(\theta)$. Wherever two surfaces meet at a shallow angle, any clearance, tolerance or offset you apply is multiplied there. The cure is to remove the vertex, not to tighten the tolerance.

LESSON — a "capsule" built as rectangle $\cup$ circles is only a stadium if the half-height equals the radius. Otherwise it is a rectangle with two bulges and a kink at each junction.

Part III — The rules, collected

#	Rule
1	An actuator's stall torque is the design load for everything it can drive to failure.
2	Match the bearing to the load direction. Thrust bearings take thrust, not moment.

#	Rule
3	Moment stiffness scales with bearing spacing squared.
4	Backlash limit cycles cannot be tuned out. Preload is mechanical.
5	For a 2-link arm, $L_2 = L_3$ removes the workspace dead zone and costs almost nothing.
6	Internal corners in a load path: $r \geq 0.5 t$, minimum 2 mm.
7	Closing an open section is the cheapest stiffness in mechanics — up to 215× in torsion.
8	Fits are properties of your machine. Measure a coupon; never assume.
9	Shrinkage is anisotropic — orient critical bores so the accurate axis defines them.
10	In bending, the outer quarter of the depth carries ~87 % of the moment: perimeters beat infill.
11	A result of exactly zero deserves suspicion.
12	Where a section is split matters more than that it is split.
13	Check the assembly bounding box against design intent before exporting — an oversized envelope means something is unmated.
14	Offsets amplify at shallow-angle vertices by $1/\sin(\theta)$. Remove the vertex, don't tighten the tolerance.
15	A capsule is only a true stadium when half-height = end radius; otherwise it has a kink at each junction.

Part IV — What was never wrong

Worth stating, because the effort should go where it pays:

- The link shells. Peak stress 0.15 MPa against a ~9 MPa derated allowable — a safety factor around 60. They were never the problem, and the topology optimisation independently confirms the box section is the right shape.
- The capsule clamshell construction. A split shell prints support-free with a flat mating face, and the split plane is parallel to the bending plane so it costs nothing in bending stiffness.
- The original 97.9/97.9 equal links. Correct, and undone by a later revision.

- The general proportions and design language. The arm looks like a robot arm and nothing like a myCobot, which was the original brief.

A per-unit number that is never multiplied out

2026-08-20 — the shaft mass. The purchasing list specified steel tube $\text{Ø}30 \times 5$ mm wall and, two lines later, described it as " ~ 93 g". Those two statements cannot both be true: that tube is 222 g at 72 mm. Meanwhile the pre-print mass gate carried the line $2 * 93$ — it counted two shafts when the arm has twelve bearings, which is six pairs, which is six joints, which is six shafts.

The two errors concealed each other. The gate reported 1144 g and passed, because it was weighing an arm with two shafts in it. The true figure with the specified steel was 2270 g — nearly double the 1.2 kg requirement, and the single largest constraint violation in the project.

Neither error was a modelling mistake. Every individual analysis was right; the shaft was simply never multiplied by its own quantity, and the spec and the mass estimate were written at different times without either being checked against the other.

What caught it: the user asking "won't that make it heavy?" — arithmetic anyone can do in their head, applied to a number I had computed precisely and then never sanity-checked.

What it changed: aluminium $\text{Ø}30 \times 2$ mm wall is now the spec (34 g each, 205 g for six, 1144 g total — the only option of five that closes the budget). The gate derives the shaft mass and count from `params.py` instead of hard-coding it, and asserts `N_SHAFT * 2 == 12` so the count can never silently disagree with the bearing count again.

The wider lesson: a bill of materials has two columns, and analysis attention goes almost entirely to the left one. Quantity is where the cheap catastrophic errors live. Any per-unit mass, cost, or power figure should be written down already multiplied out.

A second finding, worth keeping for its own sake: printing the shaft was rejected early on the assumption it would be too weak. Running the numbers showed the opposite — at servo stall a printed $\text{Ø}30$ shaft sees 0.55 MPa shear (SF 32) and 0.57 MPa bending (SF 104). It is nowhere near failing structurally. It fails for two local reasons: the M5 preload thread strips (cut across layer lines, ~ 18 MPa capacity against ~ 16 MPa demand), and the press fit creeps away because a press fit is stored elastic strain and plastic relaxes. Rejecting it for the right reason led to a better design — metal only at the thread and the bearing seats, which is what made the light tube viable.

Grip length is not boss height

2026-08-21 — the seam screws. The purchasing list specified M3 $\times$ 12 for the link seams and M2.5 $\times$ 10 for the tip ears. Both are too short to work at all.

The seam screw enters from outside, through the flat back face of the tongue half, and threads into an insert in the groove half. Its grip is therefore the full half-shell depth, 14.50 mm — not the boss height, and not the wall thickness. An M3 $\times$ 12 stops 2.5 mm short of the seam plane; it never touches the insert. Correct is M3 $\times$ 20 (14.5 grip + 5.8

insert), and M2.5 × 18 for the ears.

There is a ceiling as well as a floor: the blind hole is 7.5 deep against a 5.8 insert, so a screw longer than 22.0 mm bottoms out and jacks the two halves apart — recreating precisely the seam gap that the tongue-and-groove was added to close.

Why it was missed: the fastener table was written by looking at the parts, and a 14.5 mm half-shell looks like a thin part. The screw does not pass through a thin wall, it passes through the whole depth of the shell. Nothing in the CAD flags this, because CadQuery models the hole, not the fastener that goes in it.

The user caught it by asking a simple question about the drawing — "does the external bolt placing come on top of it?" — which forced the fastener path to be traced end to end for the first time.

General rule: a fastener schedule should record grip length, measured through the assembly, alongside the part it fastens. A length quoted without a grip is a guess.

Two tool interfaces in one machine that did not match

2026-08-21 — the missing Ø10 pilot. `cad/base_wrist.py` has defined the standard tool interface since the beginning: 4 × M3 on a Ø30 bolt circle, Ø10 centre pilot. When the wrist was integrated into three parts, `wrist_j6_output()` grew its own tool flange — and got the bolt circle but not the pilot. Two tool mounting faces in the same machine, specified the same way in prose, built differently in CAD.

Without the pilot a tool is located only by four clearance holes, so it can sit off-centre by the hole clearance — on a machine whose whole argument is about hundredths of a millimetre. The pilot also doubles as the cable route from the tool into the hollow wrist.

Worse: the isometric drawing I had just produced called out "Ø10.00 PILOT" on a part that did not have one. A drawing asserting a feature that is absent is the most dangerous kind of drawing error, because it is trusted.

The user caught it by looking at the sheet and saying the part was missing something.

Root cause, and the real fix: `output/cad/volumes.json` is the authority for part mass everywhere — the mass gate, the drawings, the URDF meshes, the BOM. Nothing regenerated it. Every script read it; none wrote it. It had been produced once by hand and went stale the moment any part changed, which is why the drawing kept quoting the pre-pilot mass, and why `fit_coupon` and `shaft_tube` were missing from it entirely. There is now a `cad/volumes.py` that rebuilds it from the STEP files and prints a diff of what moved.

General rule: any generated file that several tools read must have exactly one script that writes it, and that script must be runnable on demand. A derived file with no generator is a stale file waiting to happen.

A correction factor applied to a model that was already corrected

2026-08-21 — the infill factor. The mass gate multiplied every part's exact STEP volume by 0.55 for sparse infill. That number is right for a part modelled solid and printed with

sparse infill. Every part in this arm is modelled hollow — 2.4 mm link walls, a 3.0 mm base shell — so the STEP volume already is the wall material. Slicing a 2.4 mm wall with 4 perimeters on a 0.6 nozzle produces 2.4 mm of solid extrusion; there is no interior left for infill to save anything in.

The factor was discounting the hollowing a second time. It hid about 240 g: the gate reported 1215 g for an arm that weighs 1437 g.

Why it survived so long. It was applied uniformly, so every part looked consistent with every other part, and the total moved plausibly whenever geometry changed. A wrong factor applied everywhere produces a self-consistent table — nothing inside the numbers disagrees. It only surfaced because adding the 24 g horn adapter pushed the total past the limit and forced the model itself to be questioned rather than the parts.

The tell that should have been caught earlier: the same 0.55 was applied to parts whose own drawings specify 100 % infill — the clamps, collars and adapters. A factor that contradicts the process sheet on the same page is wrong by inspection.

General rule: a correction factor needs its assumption written next to it. 0.55 means nothing; 0.55 — sparse infill, valid only for solid-modelled parts would have been checked against a hollow part the first time someone read it. FILL = 0.90 now carries that note.

Outcome. Offered a lightening pass (~1333 g) or narrower bearings (~1148 g), the user accepted 1437 g rather than thin the link walls — bending stiffness goes as thickness cubed, and that stiffness is what the whole bearing redesign exists to protect. 1450 g is now a hard ceiling: the gate fails on it rather than warning, with 13 g of headroom.

A second correction, in the same session: I quoted a bearing swap as saving "~90 g" without computing it. The real figures are 37 g for 6805 and 185 g for 6706 — wrong in both directions. An unverified number offered as a basis for a design decision is worse than no number, because it gets acted on.

Checking the space instead of the path

2026-08-21 — the sine collision. `interference.py` samples the whole joint-limit box and reports about 30 % of poses colliding. That number had been on screen for days and was read as background: the box is enormous, most of it is nowhere the arm needs to go, and the sine task was known to work.

The task did not work. Running the same exact-mesh test on the 240 solved waypoints found 72 of them colliding — the upper arm grazing the top rim of the base pedestal by 1.27 mm. The demo that had been reported as verified would have ground plastic on 30 % of its stroke.

Two different questions. "Can the arm collide anywhere in its joint limits?" and "does this trajectory collide?" have almost nothing to do with each other. The first was being answered and quietly ignored; the second was never asked. A high number on an irrelevant test is worse than no test, because it trains you to ignore that class of result.

The fix was strictly better in every dimension, which is itself worth noticing. Raising the trace 20 mm cleared every pose and improved accuracy 14× (0.194 → 0.014 mm rms).

Both came from the same cause: at the old height the IK was working hard against the J2 limit, and a solver pinned against a limit is both inaccurate and in an extreme posture. When a fix improves everything at once, the original setting was usually not a considered trade-off — it was an accident nobody had questioned.

Two attempted fixes that did not work, kept because knowing what fails is worth as much: narrowing the pedestal top (30 % → 26 %, so the graze was not only the rim) and a collision-aware IK seed bank (found no accurate clear branch on its own). Geometry was not the problem; the trajectory placement was.

General rule: verify the thing you will actually run. Coverage of the whole state space is a different and much weaker claim than correctness of the specific path, and it is the path that will be executed.

A union that does not touch is still a success

tool_dock was modelled, rendered, dimensioned on a drawing sheet and counted in the mass budget with no latch on it. The three latch lugs — the entire reason that tool needs no seventh actuator — were placed inside the capture cone at a height where the bore had already opened to Ø25.8, so they sat 5.5 mm clear of any wall. CadQuery's .union() does not fail when the shapes do not touch; it returns a compound. Everything downstream treats a compound as a part. It would have come off the print bed as a probe plus three loose 19 mm³ crumbs.

The same class of error had already been caught once in this project — horn_adapter exported as two bodies because a box grew the wrong way — and the lesson was not generalised then. It is now: check_tool_fit.py counts len(shape.Solids()) for every tool part and fails on anything but 1, and cad/end_effector.py prints the solid count for each part as it exports.

| Boolean success is not geometric contact. Ask for the solid count.

Three defects in one interface, none of them visible

The docking pair was checked, part by part, and passed. Checked as a pair, it could not have worked at all:

1. the probe's latch lugs were not attached to the probe (above);
2. the target's capture groove was cut 3.30 mm deep into a 1.65 mm wall, which is not a groove but a parting cut — it severed the spigot tip, and the three entry slots then chopped what was left into three loose arcs;
3. even with both parts whole, the capture cone was 16 mm deep and the spigot 10 mm tall, so the cone mouth grounded on the target flange while the tip was still 6 mm short of the throat. Probe and target fouled at every insertion depth and every roll angle.

Two of those three look right in a render. The third is invisible in any single-part view, because it is a relationship between two parts, and no drawing of either part contains it.

What found them was asking the geometry questions a picture cannot answer: how many solids are you, what is left of the wall after this cut, and drive one mesh onto the

other and tell me what touches. The mate is now simulated through all five states — half inserted, seated, rolled to latch, pulled while latched (must catch), rolled back (must release) — and the capture envelope is measured by sweeping the probe sideways until it fouls, rather than quoted from the cone diameters.

A part that passes on its own tells you nothing about the joint it belongs to. Test the mate, not the parts.

A drawing that asserts a defect the part no longer has

Three three-view sheets carried a bold red banner reading NO Ø42.00 BEARING POCKET. The banner came from a hardcoded list of part names, written when the audit first found those pockets missing. They were fixed the next day. The banner went on claiming the defect — in red, on the sheets someone prints from.

This is the J6 pilot error pointing the other way. A drawing that asserts a feature the part does not have is dangerous; a drawing that asserts a defect the part does not have is the same failure, and costs the same thing: it destroys the reader's ability to trust any stamp on any sheet. The banner now reads the pocket count out of the STEP.

Never hardcode a part's status. Status is a question you ask the geometry, every time you draw it.

The same physical quantity, two different numbers, in two documents

Every drawing sheet quoted its part's mass at 55 % fill while the mass budget, the pre-print gate and every report used 90 % — so the sheets under-quoted every part by 39 %. The 0.55 was left over from before we established that a 2.4 mm wall at four perimeters and a 0.6 mm nozzle prints solid, and that applying an infill discount to geometry already modelled hollow counts the hollowing twice.

Nothing failed. Both numbers were internally consistent within their own document. That is what makes this the hard kind: it can only be caught by comparing two documents that nobody has any reason to read side by side. `mass_of()` now takes its fill from `params.MASS_FILL`, the same constant the budget uses.

If a number appears in two places, one of them is a copy, and copies rot.